HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

Project Description

HTML Quick-Styler is an intelligent web page generation platform that leverages IBM's Granite 3.3 2B Instruct model to provide comprehensive HTML/CSS page creation with professional styling and responsive design. The platform addresses the challenge of rapid web development by delivering AI-powered HTML generation, automatic section detection, modern CSS styling, color palette management, and production-ready code output.

Using Granite 3.3 2B's advanced language capabilities, the system analyzes user descriptions (page title, section requirements, color preferences), generates semantic HTML structure, applies modern CSS styling with animations, creates responsive layouts, and provides multiple color palette options. The system ensures professional output through Gradio's user-friendly interface while maintaining high-quality design standards through sophisticated template generation and AI-powered content suggestions.

HTML Quick-Styler transforms web development into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and AI-powered analysis that provides personalized HTML pages considering design trends, accessibility standards, and responsive design principles.

Scenarios

Scenario 1: Portfolio Website Generation

A freelance designer wants to create a professional portfolio website quickly and efficiently. The system first detects the project requirements and analyzes the designer's style and design preferences. Based on this understanding, it generates a complete website structure that includes a visually appealing hero section with a welcoming message and a strong call-to-action to engage visitors immediately. It also builds a responsive portfolio gallery arranged in a clean grid layout to showcase projects effectively across devices. An about section is created to present the designer's professional bio in a compelling manner, helping visitors understand their background and expertise. To strengthen credibility, the system includes a testimonials section that highlights positive user experiences and social proof. A contact form with proper input validation ensures smooth communication while preventing errors, and finally, a professional footer is added with social media links and essential information to complete the polished look of the website.

Scenario 2: SaaS Landing Page

A startup needs a powerful landing page for launching their new product. The system responds by creating a compelling hero banner that clearly communicates the product's value proposition and key messaging. It then generates a features showcase section, presenting product capabilities with visually appealing icons and concise descriptions to make the benefits easy to understand. A pricing comparison table is designed with clear plan options to help potential customers make informed decisions. To build trust and credibility, the system incorporates a testimonials section that highlights positive user experiences and social proof. Prominent and strategically placed call-to-action buttons are added throughout the page to guide visitors toward signing up or purchasing, ensuring strong conversion optimization.

Scenario 3: Corporate Website

A company requires a professional corporate website with multiple structured sections. The system delivers a clean and polished layout starting with a sticky navigation header that includes company branding and allows smooth navigation across sections. It creates a company overview section that clearly communicates the organization's mission, vision, and core values. A services grid is generated to present detailed descriptions of the company's offerings in an organized and visually structured manner. The website also includes a team members section with photos and short profiles to add a human touch and build trust with visitors. Additionally, a contact section is implemented with company information and an inquiry form for seamless communication. The site is completed with a professional footer containing privacy policies and important links, ensuring a corporate-standard presentation.

Scenario 4: Custom Color Palette Exploration

A designer wants to experiment with different color schemes to find the perfect visual identity for their project. The system generates eight carefully curated pre-designed color palettes, including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal themes. Each palette comes with clearly defined hex color codes for every component, making implementation straightforward. The system provides a real-time preview feature, allowing the designer to instantly visualize how each color scheme affects the overall design. It ensures consistent color application across all page elements to maintain visual harmony. Additionally, dark theme-optimized backgrounds are included to support modern design trends and improve user experience in low-light viewing environments.

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- **Python 3.x**: Core programming language for application logic
- **IBM Granite 3.3 2B Instruct**: AI model for intelligent content generation
- **Gradio**: Interactive web interface framework for user interaction
- **HTML5/CSS3**: Modern web standards for page generation
- **JavaScript**: Client-side interactivity and animations
- **Google Colab**: Cloud-based development environment
- **Responsive Design**: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- **Python 3.8+**: Download from python.org
- **Google Account**: Required for Google Colab access
- **Gradio**: Install via pip (pip install gradio)
- **Web Browser**: Chrome, Firefox, Safari, or Edge
- **Internet Connection**: Required for Colab and model access

Knowledge Prerequisites

- **Python Basics**: Functions, classes, dictionaries, string manipulation
- **HTML/CSS**: Understanding of HTML structure and CSS styling
- **Gradio Framework**: Basic understanding of interface components
- **Web Design**: Color theory, responsive design principles
- **Google Colab**: Familiarity with notebook interface

Hardware Requirements

- **Processor**: Intel i3 or equivalent (Colab runs on cloud)
- **RAM**: Minimum 4GB local (Colab provides GPU/TPU)
- **Storage**: 1GB free space for generated files
- **Display**: Minimum 1280x720 resolution for interface

Project Workflow

Phase 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model.

Activity 1.1: Set up Google Colab Environment

- Create new Google Colab notebook
- Enable GPU runtime for better performance
- Install required Python packages
- Verify environment configuration

Activity 1.2: Configure IBM Granite Model

- Access Hugging Face model repository
- Load Granite 3.3 2B Instruct model
- Test model inference capabilities
- Configure generation parameters

Activity 1.3: Install and Configure Gradio

- Install Gradio via pip
- Test basic Gradio interface
- Configure sharing options
- Verify interface rendering

Phase 2: Core Application Development

Build HTML generation engine and template system.

Activity 2.1: Build Color Palette Generator

- Create

ColorPaletteGenerator class

- Define 8 color palette schemes
- Implement palette selection logic
- Add color validation methods

Activity 2.2: Implement HTML Page Builder

- Create

HTMLPageBuilder class

- Build section detection algorithm
- Implement template generation methods
- Create SEO meta tags generator

Activity 2.3: Develop CSS Generation Engine

- Build comprehensive CSS generator
- Implement responsive design rules
- Add animation and transition effects
- Create mobile-first media queries

Activity 2.4: Build Section Templates

- Create hero section template
- Build features grid template
- Implement portfolio gallery template
- Add contact form template
- Create pricing and testimonial templates

Phase 3: Gradio Interface Development

Create interactive user interface with multiple tabs.

Activity 3.1: Interactive HTML Generation & Preview Interface

- Activity 3 focuses on designing and implementing a **complete HTML generation and visualization interface** that enables users to create, customize, preview, and understand web page layouts through an intuitive, multi-section workflow.

Phase 4: Google Colab Deployment

Integrate all components and perform comprehensive testing.

Activity 4.1: End-to-End Integration

- Connect all interface components
- Test data flow between tabs
- Validate generation pipeline
- Test error handling

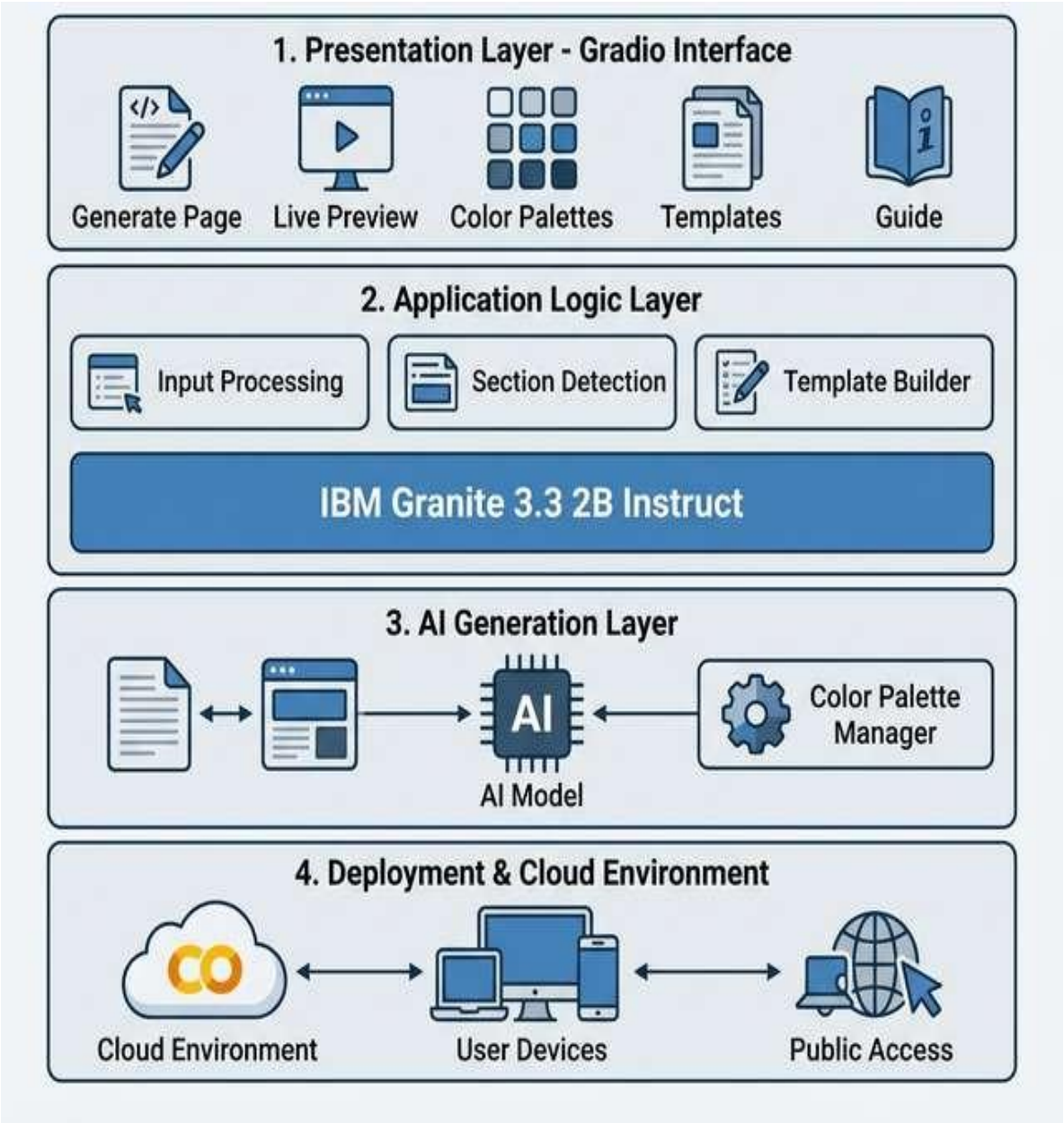
Phase 5: Deployment & Documentation

Deploy application and create comprehensive documentation.

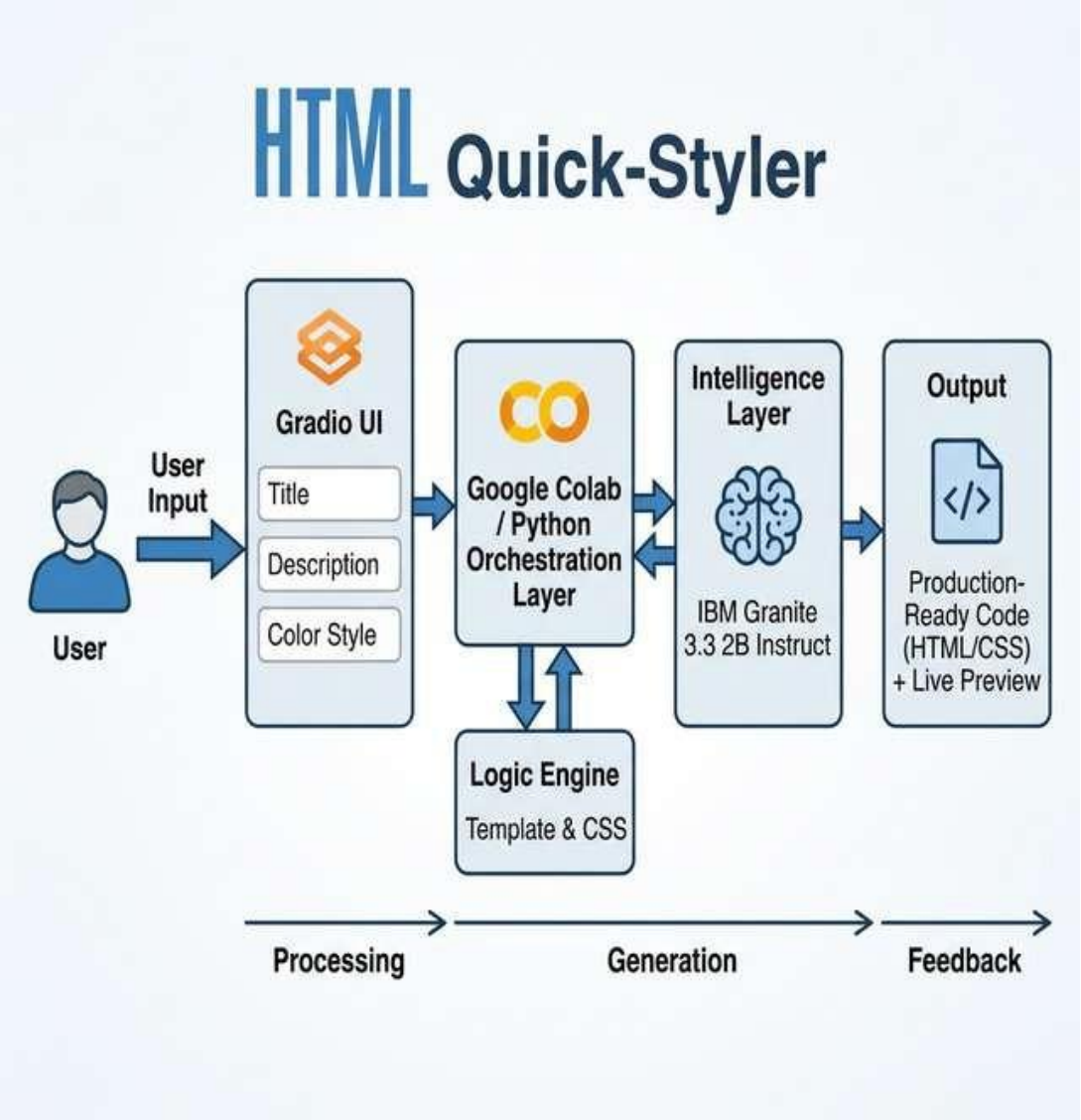
Activity 5.1: Functionality Testing

- Configure Gradio sharing settings
- Enable public access link
- Test external access
- Document deployment steps

Technical Architecture



Project Structure:



MILESTONE 1:

Environment Setup & Model Configuration Establish Google Colab environment and configure Granite model Activity

1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
import os
```

Activity 1.2: Configure IBM Granite Model

Access Hugging Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct

Colab Notebook :

```
if HAS_TRANSFORMERS and os.environ.get('USE_GPU', '0') == '1':
    print("Loading Granite Model...")
    try:
        tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
        model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
    except Exception as e:
        print(f"Failed to load model: {e}")
```

MILESTONE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces

```python
class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
                'accent': '#feca57',
                'background': '#111111',
                'surface': '#222222',
                'text_primary': '#ffffff',
                'text_secondary': '#cccccc',
                'border': '#444444'
            },
```

```
        'ocean': {
            'primary': '#009ffd',
            'secondary': '#2a2a72',
            'accent': '#00c3ff',
            'background': '#0b132b',
            'surface': '#1c2541',
            'text_primary': '#e0fbfc',
            'text_secondary': '#98c1d9',
            'border': '#3a506b'
        },
```

```
        'forest': {
            'primary': '#2d6a4f',
            'secondary': '#1b4332',
            'accent': '#52b788',
            'background': '#081c15',
            'surface': '#143601',
            'text_primary': '#d8f3dc',
            'text_secondary': '#95d5b2',
            'border': '#40916c'
        },
        'sunset': {
            'primary': '#ff7b54',
            'secondary': '#ffb26b',
            'accent': '#ffd56b',
            'background': '#1a0b0b',
            'surface': '#2d1414',
            'text_primary': '#ffffff',
            'text_secondary': '#ffdfdf',
            'border': '#5a2e2e'
        },
        'luxury': {
            'primary': '#d4af37',
            'secondary': '#aa8000',
            'accent': '#f3e5ab',
            'background': '#000000',
            'surface': '#1a1a1a',
            'text_primary': '#ffffff',
            'text_secondary': '#b3b3b3',
            'border': '#333333'
        },
```

```
        'creative': {
            'primary': '#9b5de5',
            'secondary': '#f15bb5',
            'accent': '#fee440',
            'background': '#00bbf9',
            'surface': '#00f5d4',
            'text_primary': '#ffffff',
            'text_secondary': '#e0e0e0',
            'border': '#00a3cc'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#e74c3c',
            'background': '#ffffff',
            'surface': '#f8f9fa',
            'text_primary': '#212529',
            'text_secondary': '#6c757d',
            'border': '#dee2e6'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

- Section extraction from user descriptions
- Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections
- SEO meta tags generation with Open Graph and Twitter Card support
- Individual section HTML generation with professional template

```python
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extraczt_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()

        for section_type, keywords in section_keywords.items():
            for keyword in keywords:
                if keyword in desc_lower:
                    sections.append(section_type)
                    break
```

```python
        if not sections:
            sections = ['hero', 'features', 'contact', 'footer']

        return sections

    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        return f"""
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{title}</title>
<meta name="description" content="{description}">
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
"""
```

Activity 2.3: CSS Generation Engine

The CSS generator creates comprehensive styling including:

- CSS variables for dynamic color management
- Gradient backgrounds and modern animations
- Responsive grid layouts with Flexbox and CSS Grid
- Smooth transitions and hover effects
- Mobile-first media queries for tablets and phones
- Sticky header with backdrop blur effect
- Card-based components with depth and shadow

```
@staticmethod
def generate_css(palette: Dict) -> str:
    """Generate comprehensive CSS"""
    css = f'''    <style>
    * {{
        margin: 0;
        padding: 0;
        box-sizing: border-box;
    }}

    :root {{
        --primary: {palette['primary']};
        --secondary: {palette['secondary']};
        --accent: {palette['accent']};
        --background: {palette['background']};
        --surface: {palette['surface']};
        --text-primary: {palette['text_primary']};
        --text-secondary: {palette['text_secondary']};
        --border: {palette['border']};
    }}

    body {{
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        background-color: var(--background);
        color: var(--text-primary);
        line-height: 1.6;
    }}
```

```
        },
        'creative': {
            'primary': '#9b5de5',
            'secondary': '#f15bb5',
            'accent': '#fee440',
            'background': '#00bbf9',
            'surface': '#00f5d4',
            'text_primary': '#ffffff',
            'text_secondary': '#e0e0e0',
            'border': '#00a3cc'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#e74c3c',
            'background': '#ffffff',
            'surface': '#f8f9fa',
            'text_primary': '#212529',
            'text_secondary': '#6c757d',
            'border': '#dee2e6'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

MILESTONE 3: Gradio Interface Development Activity

3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs:

```python
class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
                'accent': '#feca57',
                'background': '#111111',
                'surface': '#222222',
                'text_primary': '#ffffff',
                'text_secondary': '#cccccc',
                'border': '#444444'
            },
```

```python
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extraczt_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()

        for section_type, keywords in section_keywords.items():
            for keyword in keywords:
                if keyword in desc_lower:
                    sections.append(section_type)
                    break
```

Tab 1: Generate Page:

```python
with gr.TabItem("⚡ Generate", id="tab_generate"):
    with gr.Row():
        # Left Column: Inputs
        with gr.Column(scale=1):
            gr.Markdown("### ✨ Page Details")

            page_title = gr.Textbox(
                label="Page Title",
                value="Modern Project",
                lines=1
            )

            page_description = gr.Textbox(
                label="Page Description & Sections",
                value="hero, features, contact, footer",
                lines=4
            )

            color_style = gr.Dropdown(
                choices=["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                label="Style",
                value="modern"
            )
```

Tab 2: Live Preview

```python
# ============ TAB 2: Live Preview (Full) ============
with gr.TabItem("🖥 Full Screen Preview"):
    with gr.Row():
        back_btn_preview = gr.Button("⬅ Back to Home", size="sm")
        full_preview_btn = gr.Button("🔄 Refresh Preview", size="sm")

    back_btn_preview.click(go_home, outputs=main_tabs)
    full_preview_html = gr.HTML("<p style='text-align:center; color:#777; padding:40px;'>
     No preview loaded. Generate a page first.</p>")
```

Tab 3: Color Palettes

```python
# ============ TAB 3: Color Palettes ============
with gr.TabItem("🎨 Color Palettes"):
    back_btn_palettes = gr.Button("⬅ Back to Home", size="sm")
    back_btn_palettes.click(go_home, outputs=main_tabs)

    gr.Markdown("### Explore Color Schemes")
    with gr.Row():
        with gr.Column():
            style_selector = gr.Radio(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="Choose Theme Style"
            )
            palette_btn = gr.Button("Get Palette Colors", size="lg", variant="primary")
        with gr.Column():
            palette_display = gr.JSON(label="Palette Data")

        def show_palette(style_name):
            return ColorPaletteGenerator.generate_palette(style_name)

        palette_btn.click(show_palette, inputs=style_selector, outputs=palette_display)
```

Tab 4: Templates

```python
# ============ TAB 4: Templates ============
with gr.TabItem("📝 Templates"):
    back_btn_templates = gr.Button("⬅️ Back to Home", size="sm")
    back_btn_templates.click(go_home, outputs=main_tabs)

    gr.Markdown("### Quick Templates")

    with gr.Row():
        with gr.Column():
            gr.Markdown("#### Portfolio Website\nHero, Portfolio Gallery, About, Contact, Footer")
            portfolio_btn = gr.Button("Use Template", size="sm")

            def load_portfolio():
                return load_template_with_preview("John Doe Portfolio", "I am a designer. hero banner, show my portfoli

            portfolio_btn.click(load_portfolio, outputs=[main_tabs, page_title, page_description, color_style, code_out

        with gr.Column():
            gr.Markdown("#### SaaS Landing Page\nHero, Features, Pricing, Testimonials, Contact, Footer")
            saas_btn = gr.Button("Use Template", size="sm")

            def load_saas():
```

Tab 5: Guide:

```python
# ============ TAB 5: Guide ============
with gr.TabItem("📚 Guide"):
    back_btn_guide = gr.Button("⬅️ Back to Home", size="sm")
    back_btn_guide.click(go_home, outputs=main_tabs)

    gr.Markdown("""
    ### HTML Quick-Styler Guide

    #### 1. Enter Details
    - **Title:** The web page `<title>`.
    - **Description:** A text description of what you want. The tool will scan your prompt for section keywords.
    - **Style:** Pick from 8 predefined aesthetic color palettes.

    #### 2. Automatic Section Detection
    The engine recognizes these keywords in your description:
    - `hero` / `banner` / `landing` -> **Hero Section**
    - `features` / `services` / `benefits` -> **Features Grid**
    - `about` / `team` / `company` -> **About Section**
    - `portfolio` / `gallery` / `projects` -> **Portfolio Section**
    - `testimonials` / `reviews` -> **Testimonial Section**
    - `pricing` / `plans` -> **Pricing Section**
    - `contact` / `form` -> **Contact Form**
```

MILESTONE 4: Deployment Activity

4.1: Google Colab Deployment Launch the application in Google Colab:

```
55s
    return demo

if __name__ == "__main__":
    app = create_ui()

    print("\n✅ HTML Quick-Styler Advanced Ready!\n")
    # Using share=True as specified in requirements for Colab deployment
    app.launch(share=True, show_error=True, show_api=False)
```

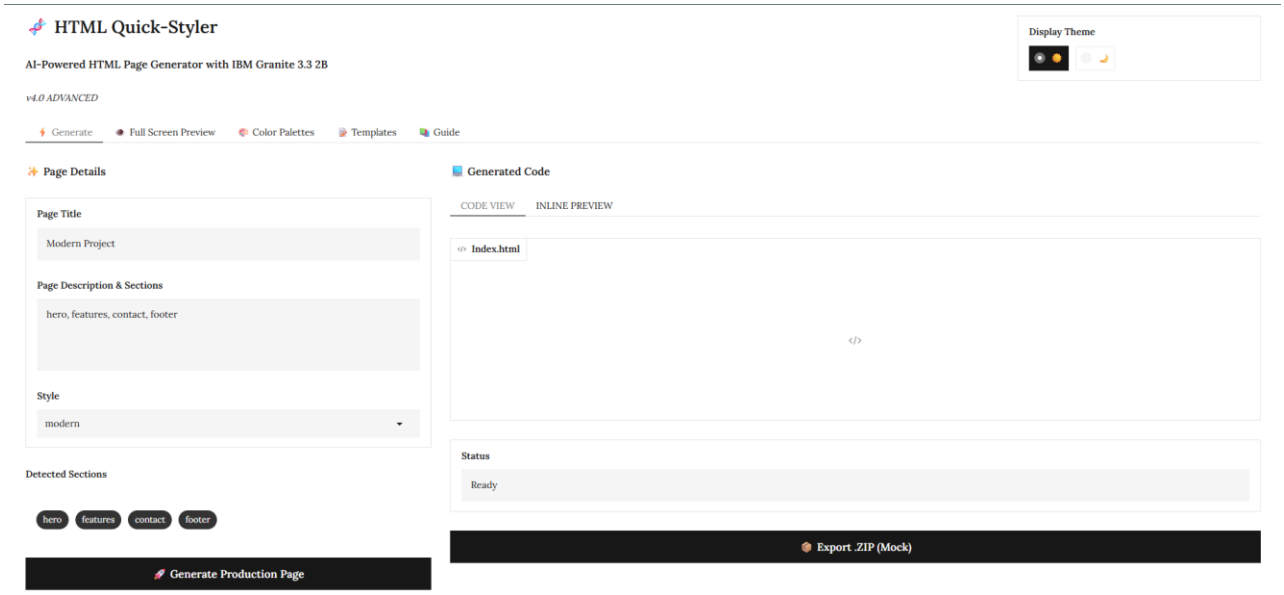
```
/tmp/ipykernel_391/773672506.py:523: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be removed in Gradio 6.0. You will need to
  with gr.Blocks(theme=gr.themes.Monochrome(), css=custom_css) as demo:
/tmp/ipykernel_391/773672506.py:523: DeprecationWarning: The 'css' parameter in the Blocks constructor will be removed in Gradio 6.0. You will need to p
  with gr.Blocks(theme=gr.themes.Monochrome(), css=custom_css) as demo:
/tmp/ipykernel_391/773672506.py:766: DeprecationWarning: The 'show_api' parameter in launch() will be removed in Gradio 6.0. You will need to use the 'f
  app.launch(share=True, show_error=True, show_api=False)

✅ HTML Quick-Styler Advanced Ready!

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://965c62ac25b7f657a7.gradio.live
```

The application will be accessible via:

- **Public URL: https://124e190506ad05ba47.gradio.live**



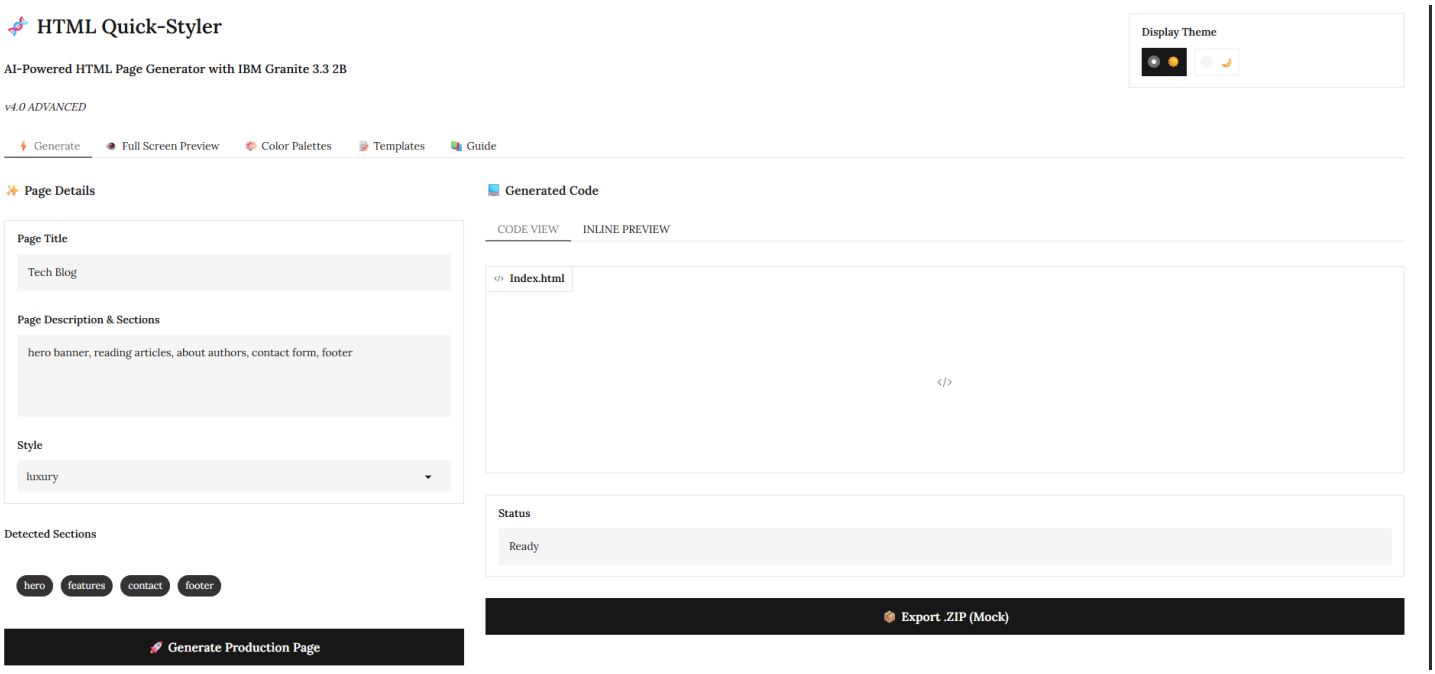
MILESTONE 5: Testing & Validation Activity

5.1: Functional Testing

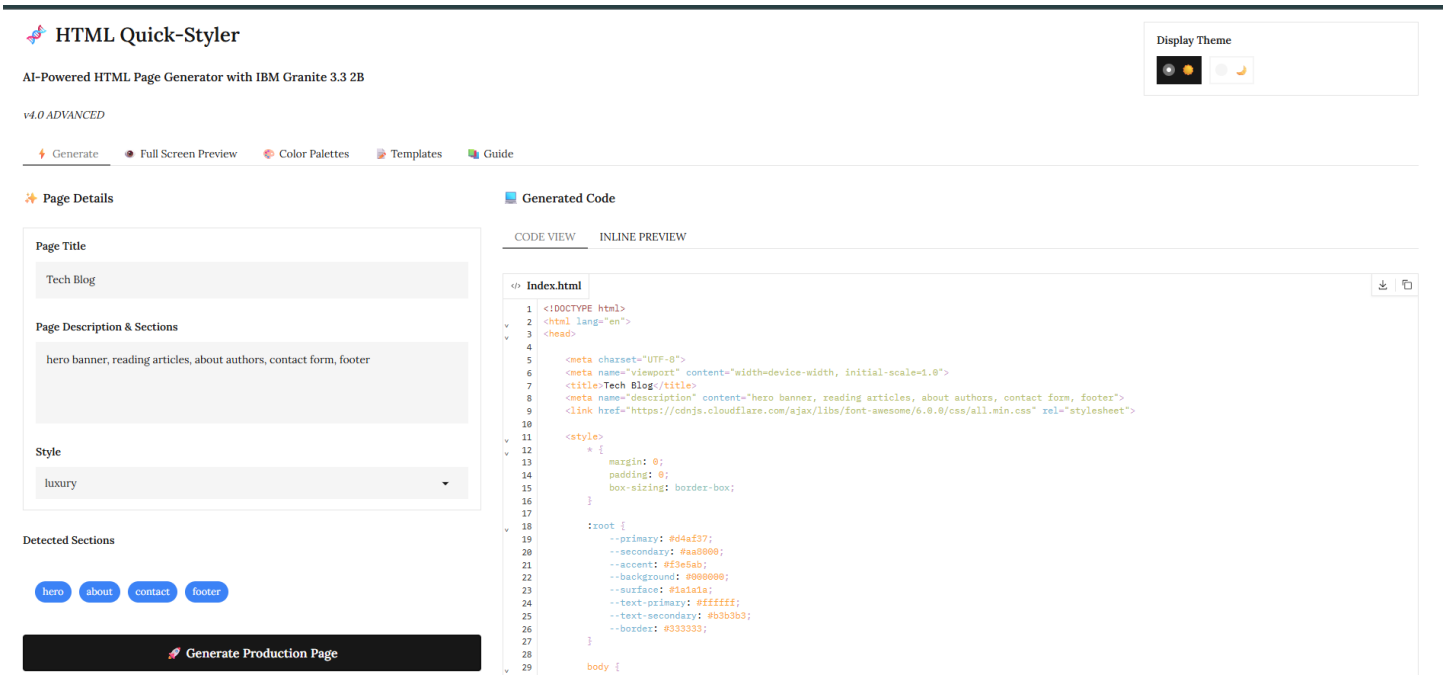
Test Case 1: Tech Blog Generation

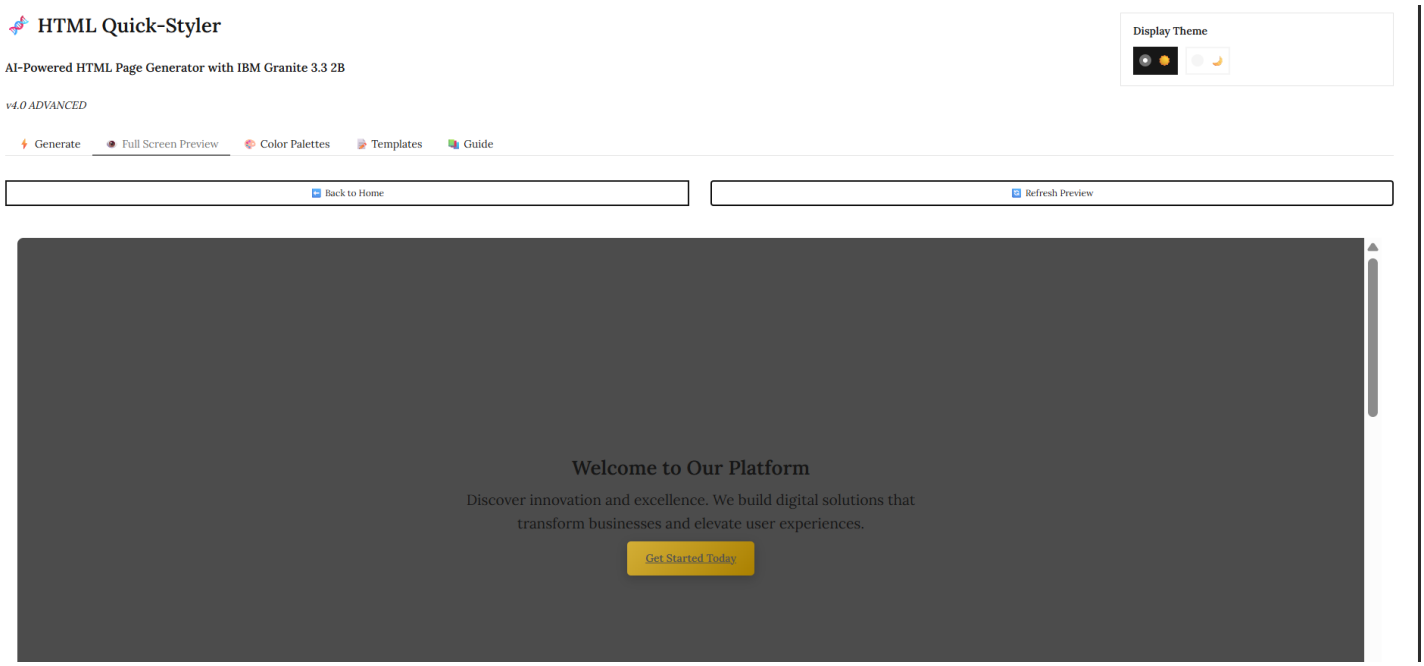
Input:

- Title: Tech Blog
- Description: hero banner, reading articles, about authors, contact form, footer
- Color Style: luxuary



Expected output:





HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

v4.0 ADVANCED

Display Theme

Generate | Full Screen Preview | Color Palettes | Templates | Guide

Back to Home

Explore Color Schemes

Choose Theme Style

modern | vibrant | ocean | forest | sunset | luxury | creative | minimal

Get Palette Colors

Palette Data

{...}

Built with Gradio | Settings

HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

v4.0 ADVANCED

Display Theme

Generate | Full Screen Preview | Color Palettes | Templates | Guide

Back to Home

Quick Templates

Portfolio Website

Hero, Portfolio Gallery, About, Contact, Footer

Use Template

SaaS Landing Page

Hero, Features, Pricing, Testimonials, Contact, Footer

Use Template

Corporate Website

Hero, About, Services, Team, Contact, Footer

Use Template

E-Commerce Store

Hero, Features, Pricing, Contact, Footer

Use Template

Blog / Magazine

Hero, About, Portfolio, Contact, Footer

Use Template

Creative Agency

Hero, About, Services, Portfolio, Contact, Footer

Use Template

🚀 HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

v4.0 ADVANCED

Display Theme

Generate | Full Screen Preview | Color Palettes | Templates | Guide

Back to Home

HTML Quick-Styler Guide

1. Enter Details

- **Title:** The web page `<title>`.
- **Description:** A text description of what you want. The tool will scan your prompt for section keywords.
- **Style:** Pick from 8 predefined aesthetic color palettes.

2. Automatic Section Detection

The engine recognizes these keywords in your description:

- `hero` / `banner` / `landing` -> **Hero Section**
- `features` / `services` / `benefits` -> **Features Grid**
- `about` / `team` / `company` -> **About Section**
- `portfolio` / `gallery` / `projects` -> **Portfolio Section**
- `testimonials` / `reviews` -> **Testimonial Section**
- `pricing` / `plans` -> **Pricing Section**
- `contact` / `form` -> **Contact Form**
- `footer` / `bottom` -> **Footer**

3. Google Colab & IBM Granite Setup

When running in Google Colab, if the transformers library is detected and memory provides, the IBM Granite model will spin up. To enable: `!pip install transformers accelerate torch gradio` Ensure you have a GPU environment (like T4) enabled in Colab!

Conclusion:

HTML Quick-Styler successfully delivers an intelligent web page generation assistant that combines intuitive design templates with IBM Granite 3.3 2B Instruct model capabilities. By leveraging Gradio's powerful interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable HTML/CSS generation accessible to designers, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted web development, making professional page creation accessible while maintaining high design standards, responsive layouts, and production-ready code quality. With eight distinct color palettes, intelligent section detection, and comprehensive template options, HTML Quick-Styler empowers users to create beautiful websites in minutes rather than hours.

Future enhancements could include custom color picker integration, JavaScript interactivity generation, component library expansion, and integration with popular web frameworks. The modular architecture allows for easy extension and customization to meet evolving user needs and design trends.